

Highlights

MacEdge: Motion-aware Collaborative Inference between Large and Small Models for Real-time Video Analysis

,

- Dynamic offloading adjusts to object motion, reducing redundant data transmission and bandwidth.
- Online 2D bin packing schedules tasks by time, boosting edge server throughput and real-time performance.
- Experiments show the framework significantly improves accuracy and latency over baseline methods.
- The system features cross-platform compatibility and a modular design for stable performance.

MacEdge: Motion-aware Collaborative Inference between Large and Small Models for Real-time Video Analysis

$a,*,1$, $b,2$

a , , , , ,
 b , , , , ,

ARTICLE INFO

Keywords:

Edge Computing
Offload Computing
Collaborative Inference
Real-time Video Analytics
Object Detection
Object Tracking

ABSTRACT

Real-time object detection is fundamental in a wide range of edge intelligence and Internet of Things (IoT) applications, including traffic monitoring, road safety management, and vehicle flow analysis. Resource-constrained end devices require lightweight and energy-efficient solutions. However, existing approaches either adopt aggressive model quantization, causing accuracy degradation, or offload computation to edge servers using high-precision models, focusing on latency but ignoring throughput limits. To address these challenges, we refine the offloading granularity from entire frames or tiles to individual objects by leveraging the outputs of tracking algorithms to guide object-specific offloading decisions. Based on this insight, we propose a **Motion-aware Collaborative Inference** framework in **End-Edge** systems (MacEdge) for real-time video analysis. On the client side, a lightweight small model dynamically detects and crops objects based on tracking results, while undetected or uncertain objects are offloaded to the edge, where a high-precision large model performs inference. On the server side, sub-images from multiple clients are aggregated for batch inference, enabling concurrent, low-latency, and high-accuracy processing. Experiments on real-time video analytics tasks demonstrate that MacEdge achieves up to 1.35× accuracy improvement and reduces end-to-end latency by up to 70.2% under 2.4 GHz Wi-Fi network and 65.1% under 5 GHz, compared with local-only inference.

1. Introduction

With the rapid growth in application demands such as congestion prediction, accident detection, and traffic violation monitoring in traffic surveillance, deploying deep neural network (DNN) models for computer vision tasks on camera-equipped end devices has become critical for enabling smart cities [1], autonomous vehicles [2], and security monitoring systems [3]. This requires devices to perform real-time object detection, tracking, and data analysis on video streams with low latency and high accuracy. However, end devices often lack sufficient computational resources to perform inference with large, high-precision DNN models. For example, deploying the YOLO 11m model on a Raspberry Pi 5, a resource-limited device, can only achieve single-digit frame rates, which is insufficient for real-time edge inference.

To address the constrained computational resources of end devices, many systems adopt End-Edge collaborative inference between end devices and edge servers [4–6]. In this setting, lightweight models on the end device process simple or motion-stable objects, whereas complex or highly dynamic objects are offloaded to the edge server for inference using a high-precision large model, thereby reducing the computation required on the end device. However, transmitting intermediate features or data introduces additional latency that can degrade real-time performance. Consequently, End-Edge collaborative frameworks require

efficient offloading strategies that minimize communication delay while improving end-to-end inference efficiency.

Latency Bottleneck in End-Edge Collaborative Video Analysis. Due to network latency, offloading every frame to the edge server is typically infeasible in real-time video analytics. Consequently, many studies offload only a subset of frames and use tracking algorithms to propagate detection results to non-offloaded frames [7, 8]. However, in existing systems, the tracker must continue predicting while awaiting the server’s detection outputs, and the detections used for recalibration often arrive several frames late. As a result, even moderate inference latency can quickly degrade tracking quality at typical frame rates, where delay-induced errors accumulate over time and ultimately reduce detection accuracy (see Section 3.1).

Limitations of Existing Solutions. Prior work typically reduces offloading latency in two ways: decreasing the amount of transmitted data or reducing offloading frequency. To reduce transmitted data, Region of Interest (ROI) and tile-based encoding lower bandwidth consumption by selectively transmitting high-quality regions while compressing the remainder [7, 9]. However, when objects are sparse, small, or misaligned with tile partitions, an entire tile still needs to be offloaded, and in extreme cases this degenerates into full-frame offloading, resulting in substantial **redundant transmission**. To reduce offloading frequency, tracking-based reuse strategies skip offloading for some frames [7, 8]. However, multi-object tracking that relies on inertial assumptions is brittle under non-uniform motion, which forces frequent recalibration and leads to **bursty offloading** and unstable latency. Meanwhile, existing edge-side batching [10, 11] often ignores client-perceived waiting time. Under multi-client contention, naive batching introduces additional queueing

*Corresponding author

 (); ()

 (); ()

ORCID(s):

1

delay, increasing the latency of returned detections and thereby making **deadline violations** more likely.

In this paper, we propose a **Motion-aware Collaborative Inference** framework in **End-Edge** systems (MacEdge) for real-time video analysis, which performs large-small model collaborative inference through dynamic offloading decisions guided by tracking outputs.

To address **redundant transmission**, MacEdge introduces an object-level offloading strategy that makes per-object decisions based on tracker outputs and object attributes (e.g., distance and effective resolution). Objects that can be recognized locally with high confidence are processed on the end device by a lightweight model, whereas hard-to-recognize objects are selectively offloaded to the edge server for high-accuracy inference using a large model. By replacing coarse-grained full-frame or ROI offloading, this strategy enables finer-grained, object-level transmission. To address **bursty offloading under non-uniform motion**, MacEdge adopts a motion-aware dynamic recalibration strategy. Specifically, it groups objects by motion patterns and adaptively adjusts the recalibration interval: motion-stable objects can rely on tracking for longer periods, whereas objects with frequently changing motion states are recalibrated more often. By reducing unnecessary transmissions, this design also prevents accuracy degradation caused by non-uniform motion. To address **deadline violations under multi-client loads**, MacEdge implements a deadline-aware batching mechanism at the edge. The edge server stitches sub-frames from heterogeneous clients into a unified canvas that matches the model input and uses an interruptible online two-dimensional bin-packing strategy to trigger inference as deadlines approach, thereby enabling efficient batching across concurrent multi-client requests.

To the best of our knowledge, MacEdge is the first dynamic, object-level End-Edge collaborative video analytics framework that jointly integrates a motion-aware object-level offloading and recalibration strategy with a deadline-aware batching mechanism, thereby sustaining real-time tracking accuracy across heterogeneous devices and dynamic network conditions. The main contributions of this paper are as follows:

- We propose an object-level, motion-aware offloading and recalibration scheme that processes easy objects locally with a lightweight model, selectively offloads hard objects to an edge large model, and adapts recalibration intervals to motion patterns to reduce redundant transmission and mitigate bursty offloading and accuracy degradation under non-uniform motion.
- We propose a deadline-aware edge batching mechanism that incorporates an online two-dimensional bin-packing strategy to reduce latency while improving throughput under multiple clients.
- Extensive experiments show that, compared with baselines such as purely local inference and ROI encoding, our framework achieves significant improvements in both accuracy and latency, and maintains

real-time inference capability across a range of frame rates and network conditions.

The remainder of this paper is organized as follows. Section 2 reviews related work; Section 3 presents our motivation; Section 4 details the proposed MacEdge framework and its analysis; Section 5 describes the implementation; Section 6 reports experimental details and results; and Section 7 concludes the paper.

2. Related Work

This section summarizes related work in three categories and discusses their limitations and differences from our approach.

2.1. Object Detection and Tracking

Detection and tracking are core research issues in video stream analysis. Object detection aims to identify and locate specific targets within video frames [12–14]. Many studies have focused on optimizing the architectures of object detection models, such as YOLO, to improve detection accuracy. For instance, Hyper-YOLO introduces hypergraph computation and hybrid aggregation networks for enhanced feature interactions [15], YOLO-MS improves multi-scale feature representations [16], and 3A-YOLO [17] increases discriminative capability via hierarchical attention mechanisms. In contrast to these model-centric approaches, our research combines detection results with a prediction-correction tracking strategy, enabling effective prediction of target motion trajectories and robust recovery from target loss or occlusion, which together enhance the real-time performance and reliability of the overall system.

Object tracking aims to continuously track the movement of targets within video sequences [18, 19]. Existing methods focus on efficient and accurate tracking to improve precision. For example, MOTIP formulates multi-object tracking as a contextual ID prediction task for direct object association [20], while OmniTrack addresses panoramic image distortions and achieves state-of-the-art performance in omnidirectional tracking [21]. In our work, we leverage high-performance tracking algorithms to reliably predict object positions and thereby reduce the frequency of data offloading. The outputs of these tracking algorithms are further used to guide offloading decisions, ensuring low latency and adaptive system behavior.

2.2. Edge Computing for Real-time Video Analytics

The primary goal of edge real-time video analytics is to utilize edge devices to perform real-time tasks such as object detection, depth estimation, or pose estimation. These tasks can be executed locally, offloaded to edge servers, or distributed across end, edge, and cloud layers for collaborative processing. Existing research can be broadly categorized into the following three approaches:

1. **On-device computation:** Methods such as FlexDNN and MCDNN reduce model size and computation

through pruning [22, 23], quantization, knowledge distillation, or dynamic sub-network selection. These approaches enable real-time video analytics on mobile or embedded devices with low latency and energy consumption, without relying on external networks, making them suitable for scenarios with unstable connectivity or high data privacy requirements.

2. **Edge offloading:** Approaches such as JSAS-MTMGS utilized DNN model sharing and partitioning on the edge [24], allowing edge servers to process diverse offloading requests from multiple clients. AccMPEG introduces a novel video encoding and streaming system that reduces end-to-end inference latency by 10%–43% without sacrificing accuracy [25].
3. **End-Edge collaboration:** Large–small model collaborative inference has emerged as an effective paradigm, enabling adaptive task partitioning between lightweight models on devices and high-precision models on edge. EdgeDuet proposed a collaborative framework where large objects are processed by local models [8], while small objects are offloaded to edge servers. AccuMO addressed multi-task scenarios by coordinating local models, local tracking, and computation offloading to enhance real-time performance through optimized scheduling [26].

Many existing studies, including ours, adopt a detection-tracking framework to ensure real-time performance [5, 7, 8, 26]. Parallel execution of detection and tracking can maximize responsiveness, but detection latency and tracker error accumulation remain challenging. To address this, we introduce modifications that enable process blocking and synchronized detection. While this may sacrifice some real-time performance, it helps alleviate errors caused by delayed results.

2.3. Communication-Efficient in Edge Video Analytics

As bandwidth remains a significant bottleneck in edge-assisted video analytics, numerous studies have focused on communication-efficient techniques to transmit compact, task-relevant features instead of full video frames.

In collaborative edge perception, prioritized information bottleneck (PIB) methods compress spatio-temporal redundancies and transmit only essential features, reducing bandwidth usage and improving detection accuracy under challenging channel conditions [27]. Accuracy-aware video encoding systems such as AccMPEG dynamically adjust encoding quality based on its impact on server-side inference, achieving reduction in end-to-end delay without compromising accuracy [25]. For latency-sensitive scenarios like AR/MR, integrated pipelines that combine low-latency offloading with fast tracking support high-accuracy object detection at high frame rates with minimal overhead [28].

Collectively, these approaches highlight the importance and effectiveness of communication-efficient designs in enabling real-time, high-accuracy edge analytics. In MacEdge, we implement object-level offloading, and design dedicated

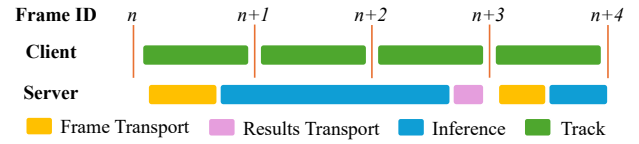


Fig. 1. Real-time video analytics using tracking, where the offloading process is independent of the tracking process, the frequency of offloading is usually fixed, and there is a delay in the results obtained by the tracker itself.

frame cropping and result processing procedures to ensure high accuracy while minimizing transmission overhead.

2.4. Split computing and layer-wise DNN partitioning

Split computing (SC) is a widely studied form of collaborative inference that partitions a single DNN at an intermediate layer, executes the front part on the end device, and offloads intermediate activations to edge or cloud server to finish inference [29, 30]. Prior work focuses on cut-layer selection and adaptation as well as reducing feature-tensor transmission overhead, e.g., via activation compression or bottleneck modules [31, 32], and recent studies extend SC to real-time object detection with adaptive runtime control [33]. A broader view of SC design choices and challenges is summarized in recent surveys [34].

While MacEdge shares the same high-level goal of optimizing latency and bandwidth via collaboration, it differs in granularity and communicated content. SC is *layer-wise* within a single model and transmits intermediate features whose size is cut-point dependent. In contrast, MacEdge adopts a *dual-model* design and performs *object-level semantic offloading*: only motion-aware cropped ROIs of selected hard objects are transmitted with full-frame fallback under abrupt viewpoint changes, while easy/motion-stable objects are processed locally. This semantic, object-level strategy aligns communication with task-relevant content and is complementary to layer-wise partitioning.

3. Background and Motivation

We propose a real-time video analytics framework based on client-edge collaboration, with particular emphasis on the design of dynamic offloading strategies on the client side and efficient batching mechanisms on the edge server. In the following sections, we outline the motivation and background from both the client and server perspectives.

3.1. Latency significantly affects accuracy

For real-time video analysis, optimizing latency is a key research focus. Due to network delays and GPU performance bottlenecks, it is almost impossible to offload every frame to the edge server in most scenarios. Many previous studies have attempted to offload a subset of frames and use tracking algorithms to estimate the results for the remaining frames. Because the offloading and tracking processes are

Table 1

Impact of inference latency on detection accuracy at different frame rates.

Frame Rate	1 Frame Delay	2 Frame Delay	3 Frame Delay
30	0.6204	0.1621	-0.1727 ¹
60	0.8407	0.6707	0.4059
120	0.9103	0.8574	0.7776

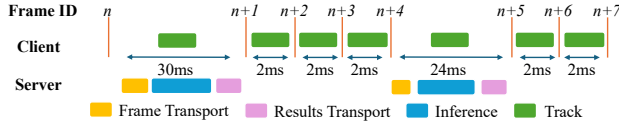


Fig. 2. Use the tracker output to make offloading decisions dynamically and calibrate the tracker with no delayed results.

independent, such methods only need to ensure that the execution time of the tracking algorithm is shorter than the frame interval to maintain real-time performance. However, since the detection results returned from the server are often delayed by several frames, the accuracy of these algorithms is often unsatisfactory. As shown in Fig. 1, the inference result for frame n requires an interval of three frames before being returned from the edge server. By frame $n+4$, even if the tracker is recalibrated with the server's result, the result itself is already three frames old and may not be accurate.

We analyzed the accuracy under different frame rates and delays using videos extracted from a 120 fps source. As shown in Table 1, for videos at 30 or 60 fps, the most commonly used frame rates, even a single-frame delay causes rapid accuracy degradation. Although 120 fps video is less affected by a single-frame delay, its short frame interval means that more frame intervals are typically needed to retrieve results from the server. The accuracy of tracking algorithms is also significantly influenced by the precision of detection results, and errors caused by delays further compound tracking errors in subsequent frames, severely impacting overall accuracy. In non-interactive scenarios like traffic monitoring and live streaming, latency tolerance is higher, often allowing for tens of milliseconds of delay without requiring strict real-time analysis. By adopting dynamic decision-making to reduce offloaded data, transmission and inference speeds can be improved, making low-latency real-time video analysis feasible for these scenarios.

¹The negative value in Table 1 appears because the accuracy metric is based on MOTA. Under delayed conditions, the total number of tracking errors, including false positives, false negatives, and identity switches, exceeds the total number of ground-truth targets. In low frame rate scenarios (30 fps) with a three-frame delay, the results returned from the edge server are approximately 100 ms outdated, causing the tracker to update using obsolete detection information. This leads to frequent association failures, resulting in a large increase in both false positives and false negatives. Consequently, in the MOTA formula, the numerator term (FN + FP + IDSW) surpasses the denominator (GT), yielding a negative value.

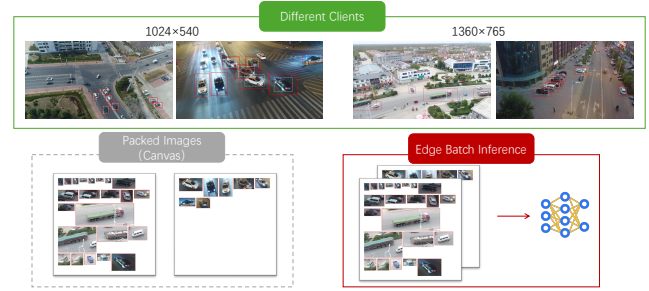


Fig. 3. After different clients send requests, the edge server packages the requests for aggregation and batch inference.

3.2. Offloading decisions can be made dynamically for each individual detected object

Currently, most devices are equipped with NPUs, which can run lightweight models on the edge with low power consumption and perform some analytical tasks. Only a portion of the objects need to be offloaded to the server for high-precision inference. Most algorithms currently either offload the entire frame or the ROI. We have found that if the decision algorithm can precisely control offloading at the object level, it can significantly reduce network transmission delays, alleviate the server load, and improve the overall system throughput. For objects moving at a roughly constant speed, edge-side tracking algorithms can be used for reliable tracking, thus reducing the need for frequent offloading. On the other hand, objects with frequent changes in motion states should be offloaded to the server.

We designed a new workflow, as shown in Fig. 2. In scenarios with relaxed real-time requirements, offloaded frames wait for server results before processing subsequent frames, alleviating accuracy degradation due to latency. The scheduler evaluates the tracking condition for each object to achieve more precise offloading decisions and reduce the volume of offloaded data. Additionally, calibrating the tracker with the latest detection results significantly improves tracking accuracy.

3.3. Model performance is influenced by frame resolution

Because the model input size is fixed, frames must be re-sized if their resolution exceeds the model's supported input size, often leading to a significant drop in inference accuracy. We evaluated three YOLO models with different numbers of parameters and input resolutions on 1080p video. Results in Fig. 4 show that even with the same number of parameters, models with different input resolutions produce significantly different inference results.

Given the wide range of resolutions from different devices, a single edge server model cannot ensure consistent input accuracy. Deploying only high-resolution models wastes resources on low-resolution requests. In our implementation, as shown in Fig. 3, upon receiving client requests, the edge server stitches together sub-images of different sizes to fit the model's input size, thereby ensuring relatively consistent

inference accuracy across clients. We refer to each stitched image aggregating multiple client sub-images as a *canvas*.

3.4. Batch processing can improve throughput

Previous frameworks have not fully leveraged GPU parallelism and typically serve only one client per server. As the number of clients increases, delays occur. Batch processing, in which several frames are processed together, significantly reduces the average inference time per frame compared to processing frames individually.

Using 1080p video, we compared single-frame and batch inference with a batch size of 8. As shown in Fig. 5, batch processing requires roughly half the time of individual processing for the same number of frames. However, batching causes the first-arriving requests to wait longer for results. Therefore, measures must be taken to ensure fairness.

We assign each client a deadline to guarantee timely results. During sub-image stitching, we use an online bin-packing algorithm, which allows the process to be interrupted at any time for immediate inference. When the server load is high, multiple large images are stitched for batch inference. Thus, our framework not only leverages batching to improve throughput but also maintains a reasonable level of fairness.

4. System Model

Based on the above considerations, we designed an Edge-Enhanced Framework for Dynamic Adaptive Offloading and Real-time Detection in Video Analytics. The system architecture is illustrated in Fig. 6.

4.1. Design Goals

We will explain our design from the perspective of the client and the server. MacEdge reuses standard detection and tracking modules and focuses on system-level mechanisms for edge-device collaboration. The on-device detector and tracker, ROI cropping and encoding, network transport, and the edge-server detector follow common implementations, while our contribution lies in the coordination logic and scheduling. On the client side, the client dynamically classifies objects for offloading, constructs motion-aware ROIs with redundant-region merging, and triggers full-frame offloading with tracker reset and re-initialization when viewpoint changes invalidate cached ROIs. On the server side, the server stitches sub-images from multiple clients and

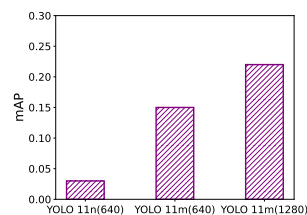


Fig. 4. Impact of Model Parameter Count and Input Size on Accuracy.

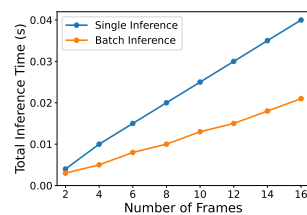


Fig. 5. Time consumed by individual inference vs. batch processing.

performs batch processing to improve GPU utilization and throughput while meeting tight return deadlines. In summary, our design goals are as follows:

- Maximize system throughput under limited GPU and network resources, enabling a single edge server to serve multiple terminals effectively.
- The client uses a modular design and enhances the parallelism of the modules to improve program operation efficiency.
- Offloading decisions should be made quickly and efficiently, without introducing significant extra computation or energy overhead on the client.

4.2. Client Model

The client consists of a Local Detector, Tracker, Offload Manager, and buffered message queues that interconnect these modules. The modular design enables adaptation based on device capabilities and is suitable for various scenarios.

4.2.1. Multi-Buffer Queue Design

Due to differences in frame content, offloading strategies, and system load, processing times vary significantly across different frames. Additionally, the execution times of client modules differ. Thus, inspired by the classical *Producer-Consumer* model, we established independent Frame Queues, Local Result Queues, and Rendering Queues.

- **Frame Queue:** Stores frames pending processing, accommodating varying data source transmission rates, such as real-time steady camera feeds or faster batch reads from storage devices.
- **Local Inference Queue:** Extracts frames from the frame queue, processes them locally, and stores results in the Local Result Queue, enabling asynchronous execution of local inference and remote offloading.
- **Rendering Queue:** Helps manage inconsistent processing times across frames, allowing the rendering thread to independently render frames at the desired frame rate.

In the scenario we envision, all these processes can run in different devices. For example, data acquisition through multiple cameras, and then a centralized computing device is responsible for analyzing each of these video streams to give the results. The video streams and analysis results are aggregated to a rendering device for unified rendering.

4.2.2. Dynamic Decision for Full-Frame Offloading Based on Viewpoint Changes

MacEdge operates in two complementary modes. By default, it performs object-level offloading, where only tracked objects are cropped and transmitted. When a sudden viewpoint change is detected, MacEdge temporarily switches to full-frame offloading to refresh the global scene and recover tracking.

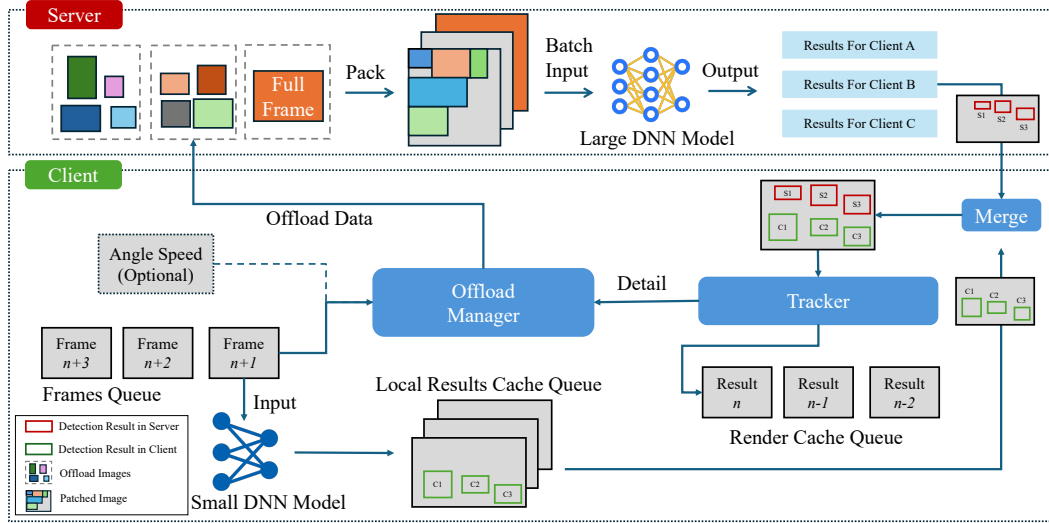


Fig. 6. An overview of MacEdge, divided into two parts: Server and Client. Client contains 3 Message Queues, an Optional Local Detector module, Offload Manager and Tracker. Server receives requests from multiple clients, batch-processes them, and then distributes them.

Trigger logic and priority are defined as follows. If the viewpoint-change score exceeds a threshold, full-frame offloading is triggered immediately and takes precedence over object-level offloading for that frame. When full-frame offloading is triggered, the tracker is reset by clearing all existing track states and ROI caches, and then reinitialized using the new full-frame detection results returned by the remote model. After reinitialization, MacEdge returns to

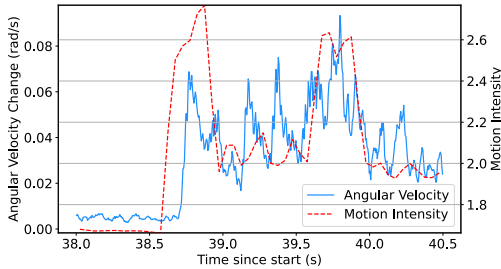


Fig. 7. Comparison of our designed program based on optical flow detection with changes in angular velocity.

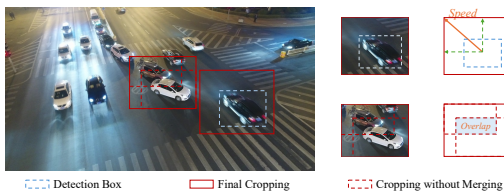


Fig. 8. Schematic diagram of the cropping scheme. In one case, the two cropping areas overlap a lot, so they are combined into one diagram, and in the other case, the cropping size and bias are determined according to the moving speed of the target.

object-level offloading in subsequent frames, using the refreshed track set and ROIs. This design ensures that full-frame offloading serves as a recovery mechanism for object-level offloading, preventing long-term error accumulation after viewpoint disruptions.

4.2.3. Cropped Region Segmentation

For all frames except the initial one, the cropped regions at frame t are generated from the tracker states of frame $t - 1$. The client first produces one crop per offloaded object using the Kalman-filter predicted box and a velocity-aware expansion margin. The expansion is applied independently on the x and y axes, so fast motion along one axis only enlarges the crop on that axis, while the crop still follows the motion direction as shown in Fig. 8. Next, if expanded regions overlap, the client merges them to remove redundant pixels and records how many objects are contained in each merged region. Finally, the client stores the bottom-left pixel coordinate of every final region for reconstruction on the original frame and encodes each region losslessly to preserve the original resolution for remote detection. Algorithm 1 summarizes the procedure.

4.2.4. Tracker Design

We selected the SORT due to its simplicity and real-time efficiency [19]. SORT utilizes Kalman filtering and the Hungarian algorithm for tracking multiple targets efficiently. However, its ability to handle complex nonlinear motion is limited, resulting in tracking errors during prolonged occlusions or rapid movements.

The client mitigates these limitations through dynamic offloading decisions, maximizing accuracy while reducing processing load.

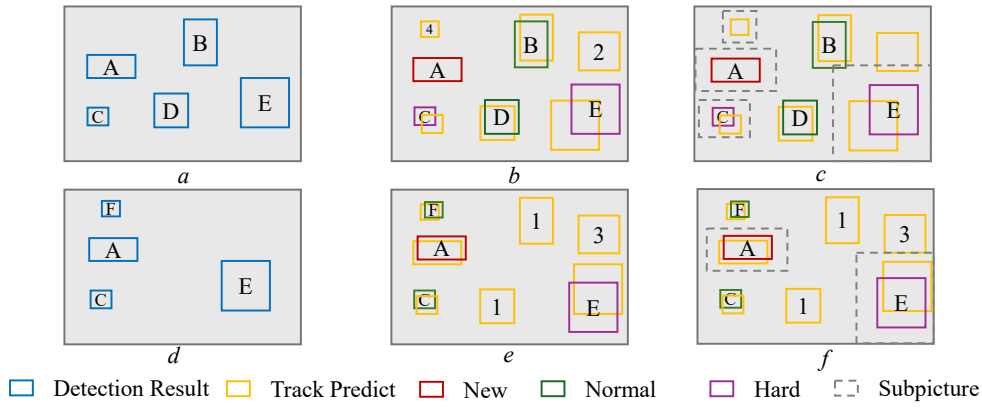


Fig. 9. Example of the offloading process for 2 consecutive frames. The detection results are labeled with the target number and the age of the tracker is labeled in the tracker. **Figure a** represents the results received from the remote server. **Figure b** represents the matching results of the Hungarian algorithm and uses the matching results to categorize the detection results. **Figure c** represents the offloading decision based on the policy. New targets, targets that are difficult to track and targets that reach the age threshold are cropped into sub-images. **Figures d and e** illustrate the offloading decision process for the next frame.

4.2.5. Tracker Update and Offloading Decision

Upon receiving remote results, the cache manager merges local and remote results according to Intersection over Union (IoU), updating the tracker accordingly. Tracker outputs feed into the rendering module and influence subsequent offloading decisions. Local results are prioritized over remote ones during **full-frame offloading** or when sub-region expansions significantly overlap, minimizing server load and latency. The tracker's Age and IoU attributes inform offloading decisions, categorizing objects as newly detected, difficult to track, or normally tracked, each with tailored strategies. Fig. 10 illustrates the conversion relationships for the three different categories of objectives.

- **Newly Detected Objects:** Continuously offloaded for four frames to allow Kalman filters to stabilize. If the IoU between the 4th frame prediction and the remote detection result is within the threshold range, these objects are reclassified for normal tracking.
- **Hard-to-Track Objects:** Continuously use remote inference results until IoU thresholds are met. Most of these targets do not have any movement rules, which makes it difficult for the tracker to output correct results, such as a turning vehicle.
- **Normally Tracked Objects:** Offloaded periodically to calibrate trackers and validate accuracy. If the IoU

between the predicted result and the detected result is significantly different, it is determined to be a Hard-to-Track Object. If the predicted result still falls within the threshold range, it is calibrated using the remote result and the calibration is still checked at a fixed frequency.

For clarity, Fig. 9(a) illustrates detected objects. Based on the Hungarian algorithm's IoU matching and the tracker's Age attribute, objects scheduled for offloading are identified. Assuming a forced offloading interval of four frames as in Fig. 9(b), normal objects not offloaded in the previous frame result in tracker mismatches, with updates occurring at intervals of 2 and 4 frames prior. Object A lacks tracker matches and is deemed new, while objects B and D match successfully with high IoU, categorized as normal tracking. Objects C and E have low IoU matches and are categorized as difficult-to-track. Fig. 9(c) shows cropping based on Kalman filter velocities. Fig. 9(d) presents new detections, and Fig. 9(e) indicates updated tracker timestamps for B and D. Object A's tracker, newly initialized, cannot predict movement, whereas object E's non-uniform motion hampers accurate prediction. Finally, Fig. 9(f) highlights new offloading decisions for new object A and difficult-to-track object E.

4.3. Server Model

The server handles multiple simultaneous client requests. It can stitch images into batches for batch inference, optimizing throughput.

4.3.1. Multi-Client Stitching and Coordinate Mapping

In a multi-client scenario, servers pack sub-images from all clients into shared canvases, rather than allocating a separate canvas per client. This approach avoids batch imbalance caused by varying numbers of sub-images from different clients. Each sub-image is assigned a unique identifier, ensuring correct association of inference results. To maintain

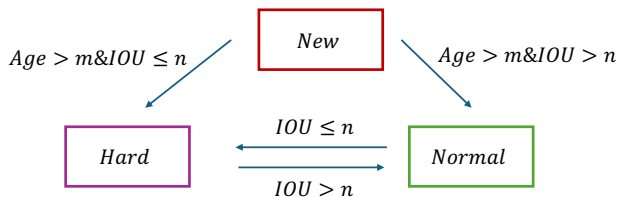


Fig. 10. Transition relationships between different categories of objects.

Algorithm 1 Client-side Cropped Region Segmentation

Input:

Offloaded object set: O ; Image size: W, H
 Predicted box for each object: $\hat{B}[i]$
 Velocity components for each object: $U_x[i], U_y[i]$
 Cropping parameters: $m_{x0}, m_{y0}, k_x, k_y, m_{\min}, m_{\max}$
 Merging threshold: τ

Output:

Final crops: C ; Bottom-left positions: P
 Merged object counts: N ; Member sets: M

Initialization:

Initialize disjoint-set D over O
 Initialize map R over O

Phase 1: Dynamic Cropping

for all $i \in O$ **do**

$m_x \leftarrow \text{clip}(m_{x0} + k_x \cdot |U_x[i]|, m_{\min}, m_{\max})$
 $m_y \leftarrow \text{clip}(m_{y0} + k_y \cdot |U_y[i]|, m_{\min}, m_{\max})$
 $R[i] \leftarrow \text{Clip}(\text{ExpandXY}(\hat{B}[i], m_x, m_y), W, H)$

end for

Phase 2: Redundancy Merging

for all $i, j \in O$ such that $i < j$ **do**

if $\text{IoU}(R[i], R[j]) \geq \tau$ **then**
 $\text{Union}(D, i, j)$

end if

end for

Phase 3: Position Mapping and Image Encoding

$C \leftarrow \emptyset, P \leftarrow \emptyset, N \leftarrow \emptyset, M \leftarrow \emptyset$

for all $G \in \text{Groups}(D)$ **do**

$r \leftarrow \text{UnionBox}(\{R[i] \mid i \in G\})$
 $C \leftarrow C \cup \{r\}; P \leftarrow P \cup \{\text{BottomLeft}(r)\}$
 $N \leftarrow N \cup \{|G|\}; M \leftarrow M \cup \{G\}$

end for

atomicity, if a client's sub-images cannot be fully included in the current batch, the entire set is deferred to the next batch. The full process is summarized in Algorithm 2.

We define three coordinate systems: (1) the full-frame $F \subset \mathbb{R}^2$ (resolution $W \times H$), (2) client-cropped sub-images S_j , and (3) server-side canvases C_k . Each S_j is a crop of F at offset $\mathbf{o}_j = (o_{x,j}, o_{y,j})^\top$ and is placed in canvas C_k at offset $\mathbf{s}_j = (s_{x,j}, s_{y,j})^\top$. The coordinate mapping is a composition of two translations:

$$\mathbf{p}_{S_j} = T_{-\mathbf{o}_j} \mathbf{p}_F \quad (1)$$

$$\mathbf{p}_{C_k} = T_{\mathbf{s}_j} \mathbf{p}_{S_j} \quad (2)$$

where $T_{\mathbf{a}}$ denotes translation by $\mathbf{a}$. Thus,

$$\mathbf{p}_{C_k} = T_{\mathbf{s}_j - \mathbf{o}_j} \mathbf{p}_F \quad (3)$$

As only translations are involved, detection box sizes are invariant.

After inference, detected boxes $\hat{\mathbf{b}}_{C_k}$ are mapped back by reversing the translations:

$$(x_F, y_F) = (x_{C_k} - s_{x,j} + o_{x,j}, y_{C_k} - s_{y,j} + o_{y,j}) \quad (4)$$

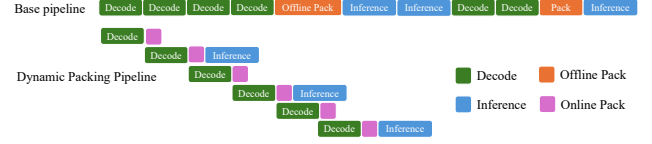


Fig. 11. Real-time inference with an online packing algorithm. Unlike offline packing, it does not need to wait for all transfers to complete.

4.3.2. Batch Processing and Parallel Computing

Traditional batch inference pipelines often employ off-line packing algorithms, where all incoming sub-images are decoded and collected before being packed into canvases for inference. If the total number of sub-images exceeds the batch size limit, multiple inference rounds are required, resulting in increased overall latency.

To address this, we adopt an online dynamic packing algorithm that does not wait for all sub-images to arrive. As shown in Fig. 11, the online approach favors immediate responsiveness over optimal packing efficiency. Inference is triggered as soon as a canvas is full or when specified deadline conditions are met, reducing wait times compared to offline methods.

Clients specify their maximum tolerable wait times with each upload, reflecting the urgency of their tasks (e.g., a client processing 120 FPS video requires lower latency than one at 30 FPS). The server monitors these deadlines and initiates inference immediately when either a canvas is full or any sub-image approaches its deadline, ensuring timely processing for all clients.

Because the sub-images are of approximately uniform size, the server can reliably estimate both packing and inference durations for each canvas. Leveraging these estimates, we design a batch scheduling algorithm that guarantees the earliest deadline is always met, as detailed in Algorithm 3.

5. Implementation

In this section, we introduce implementation details of the system, **including model selection and specific implementation details.**

5.1. Edge Server Implementation

5.1.1. Large Detection Model on Edge

We selected a YOLO 11m model with an input size of 640×640, pretrained on the VisDrone dataset, and converted to the TensorRT format [35, 36]. The maximum canvas batch size was set to 8 for batch inference.

5.1.2. Inference Data Processing

Each client is assigned a unique identifier upon connection. All client requests are enqueued into a shared task queue, with read–write locks ensuring atomic batch operations. Upon dequeuing, image data are decoded and packed into canvases using a simple online packing strategy. Specifically, we employ the Next Fit algorithm for its efficiency, noting that optimal online 2D bin-packing is outside the

scope of this work. For scenarios with highly variable sub-image counts and sizes, more sophisticated online algorithms could be adopted to further increase throughput. A 2-pixel gap is reserved between adjacent sub-images during packing to prevent edge interference in detection.

5.1.3. Inference Result Processing and Distribution

During inference, the number of objects detected in a packed sub-image may exceed expectations. In such cases, we first filter out low-confidence detections. We then match detections based on object size similarity (with a threshold of 80% for size matching) and spatial proximity, preferentially pairing detections of similar size. From these candidates, we select the final detection results based on the minimum distance to avoid duplicates. Each client is allocated an independent result queue, which accommodates differences in network speed and latency by buffering results before sending them back to the respective client.

5.2. Client Implementation

5.2.1. Local Tracker Implementation

We used C language to implement a local tracker based on the SORT algorithm [19]. The Hungarian algorithm for assignment was adapted from an open-source C++ implementation originally derived from a *MATLAB* library, and the Kalman filter was implemented using the *GSL* library. Memory management for the tracker is handled in a C program, and we provide interfaces for interaction via both Python and Swift. This ensures that the tracker can be used on both desktop and mobile clients.

Algorithm 2 End-Edge Collaborative Inference

Input: Full-frame image $F \in \mathbb{R}^{W \times H}$

Output: Final detections $\hat{B}$ on F

1. Client:

- a. Determine crop boxes $\{\mathbf{o}_j, w_j, h_j\}$.
($\mathbf{o}_j = (o_{x,j}, o_{y,j})$: crop origin; w_j, h_j : width/height)
- b. **For each** j :
 $S_j \leftarrow$ crop F at offset $\mathbf{o}_j$ with size (w_j, h_j)
 Send $(S_j, \mathbf{o}_j)$ to server.

2. Server:

- a. Tile S_j into canvases $\{C_k\}$ at offsets $\mathbf{s}_j$.
- b. **For each canvas** C_k :
 $D_k \leftarrow$ run detector on C_k
For each detection $d \in D_k$ with box (x_{C_k}, y_{C_k}, w, h) :
 Identify j such that (x_{C_k}, y_{C_k}) falls in S_j region.
 $x_S \leftarrow x_{C_k} - s_{x,j}$, $y_S \leftarrow y_{C_k} - s_{y,j}$
 Emit record $r = (j, x_S, y_S, w, h, \text{score}, \text{class})$
- c. Send all records to client.

3. Client:

- a. **For each record** $r = (j, x_S, y_S, w, h, s, c)$:
 $x_F \leftarrow x_S + o_{x,j}$, $y_F \leftarrow y_S + o_{y,j}$
 Add (x_F, y_F, w, h, s, c) to final set $\hat{B}$
 - b. Return $\hat{B}$
-

Algorithm 3 Batch inference with guaranteed shortest deadline

Input:

- Canvas size: $\{H, W\}$
- Max canvases per inference round: L
- Packing-time estimator: t_{pack}
- Inference-time estimator: t_{inf}
- Decoded sub-images (ordered by deadline) queue: Q

Output:

- Inference results

Initialization:

- $\text{openCanvas} \leftarrow$ new empty canvas
- $\text{readySet} \leftarrow \emptyset$

while True do

- if** Q empty **then**
 continue

- end if**

- $r \leftarrow Q.\text{pop}()$

- if** r not fit openCanvas **then**

- $\text{readySet} \leftarrow \text{readySet} \cup \{\text{openCanvas}\}$

- $\text{openCanvas} \leftarrow$ new empty canvas

- end if**

- place r into openCanvas

- $\tau \leftarrow \min(\text{deadlines in } \text{openCanvas} \cup \text{readySet}) - \text{now}()$

- $\text{remain} \leftarrow t_{\text{pack}} + t_{\text{inf}}$

- $\text{urgent} \leftarrow (\tau \leq \text{remain})$

- $\text{full} \leftarrow (|\text{readySet}| + 1 == L)$

- if** urgent or full **then**

- $\text{readySet} \leftarrow \text{readySet} \cup \{\text{openCanvas}\}$

- INFERENCE(readySet) // Start Inference Process

- $\text{openCanvas} \leftarrow$ new empty canvas

- $\text{readySet} \leftarrow \emptyset$

- end if**

- end while**

5.2.2. Small Detection Model on Local

For on-device inference, we employed a YOLO 11n model with an input size of 640×640, also pretrained on the VisDrone dataset. The model was quantized to INT8 precision to reduce computational load on client devices. On our Swift-based iOS client, we use the Core ML framework for local inference, while the Python-based client uses the TensorRT engine.

5.2.3. Network Communication

We adopted a socket-based communication protocol between the client and server, utilizing persistent connections to minimize the overhead of repeatedly establishing new connections. Each offloaded image (or sub-image) is first encoded in Base64 format using OpenCV, and then packaged together with its metadata into a JSON payload for transmission. This approach ensures uniform data formatting and reduces transmission latency.

6. Evaluation

In this section, we present the experimental setup, including dataset selection, evaluation metrics, and comparison methods.

6.1. Experiment Setup

6.1.1. Dataset

We evaluate the offloading strategy on VisDrone and UAV Benchmark M. VisDrone provides human-annotated bounding boxes, and we report accuracy against these annotations. UAV Benchmark M has no manual labels, so we use the edge-server model outputs as the reference and measure the accuracy degradation of the offloaded pipeline relative to the original remote model under the same reference. To study different frame rates, we interpolate the 30 FPS videos to 60 FPS and 120 FPS using Super-SloMo [37] and generate the reference outputs for each frame rate accordingly.

For validating the full-frame decision strategy, we record synchronized video and gyroscope signals using an iOS tool we developed. We captured a 30-second video at 720p and 30 FPS with abrupt viewpoint changes.

6.1.2. Comparative Methods

We compare our approach with several representative baselines and ablations, including local-only inference, ROI encoding, and variants of our proposed method. For fair comparison, all methods adopt consistent offloading intervals and experimental settings.

- **Local Only:** All inference is performed on the local device, with no data uploaded to the edge server.
- **ROI Encoding:** Following the EAR method and other related research [7, 9, 38], each frame is partitioned into ROIs and non-interest regions based on the previous frame's detections. ROIs are transmitted with high quality, while the rest are heavily compressed, reducing transmission bandwidth. Tracker calibration is ensured by consecutively offloading two ROI-encoded frames and periodically offloading an uncompressed full-frame image to avoid tracking drift and capture new objects. Based on AccMPEG's research, we selected H.265 as the encoding method [25].
- **Edge Only:** Our offloading and cropping pipeline is applied to every frame, using prior detection results for cropping, but without any dynamic or tracking-based decisions. All detected objects are always offloaded to the server.
- **No Local:** Local inference is disabled in our pipeline, simulating low-performance or power-constrained devices. All detection is performed remotely, to evaluate end-to-end accuracy and latency.
- **No Tracker:** This solution does not introduce a tracking algorithm, **so it cannot provide dynamic perception data, and the tracked object will be offloaded in each frame.**

For all methods, the offloading intervals (including the ROI and full-frame intervals for ROI encoding) are set consistent with our proposed approach to ensure comparability.

6.1.3. Experimental Parameters

The experimental parameters we set are as follows. For some parameters not mentioned, please refer to our open source code. We set these parameters based on experience, but there is still room for optimization and adjustment based on the common application and requirements.

- **Local and edge detection confidence threshold:** 0.75. Detections below this threshold are discarded but may still be detected by the edge server.
- **Local-remote result IoU merge threshold:** 0.8. If two results have an IoU greater than 0.8, they are considered the same object, with priority given to local inference results to reduce server load.
- **Local inference interval:** Based on Core ML inference time on iPhone, for 30 FPS and 60 FPS videos, local inference time is less than the frame interval, so inference is performed every frame. For 120 FPS, local inference time exceeds the frame interval, so inference is performed every other frame, using tracking to predict results for intermediate frames.
- **Tracker parameters:** IoU threshold for association during the matching stage is set to 0.5. Different maximum tracker age values are set to accommodate differences in object speed and detection update frequency at different frame rates. For 30 FPS and 60 FPS, a tracker is deleted if it fails to be updated for 5 consecutive frames. For 120 FPS, the threshold is extended to 10 frames to avoid prematurely discarding valid trackers at high frame rates.
- **Motion intensity threshold:** 15. **The motion-intensity estimator outputs a score in real time.**
- **Edge forced-offloading interval:** A strategy to forcibly offload common objects for calibration and status assessment. Set to every 4 frames for 30 FPS and 60 FPS, and every 8 frames for 120 FPS.
- **Initial sub-image cropping size:** The length and width are both expanded to twice the original detection result.

6.1.4. Experimental Environment

We chose several different types of devices for experiments, all being typical consumer-grade products to reflect the method's performance on diverse devices.

- **Edge Server:** AMD Ryzen 5 9600X, NVIDIA RTX 4060 Ti (16GB), 32GB RAM, running Linux Mint 22 with Python 3.12.
- **Client:** The Wi-Fi network environment experiment was conducted using a single PC with the following

configuration: Intel i3-12100F, NVIDIA RTX 4060, 32GB RAM, running Debian 12 with Python 3.12. This was used for collecting baseline data. The experiment in a 5G cellular network environment was conducted using an iPhone 15 Pro running iOS 18.

- **Network:** Devices connected through a Redmi AX1800 router. The edge server used a 1 Gbps Ethernet link; other devices connected via 2.4 GHz or 5 GHz Wi-Fi. Network bandwidth was controlled using Linux tc to simulate various conditions.

For multi-device tests, to control variables, we collected offloading logs and content under different conditions using the above client, and simulated sending from 3 Raspberry Pi 5 and 3 Raspberry Pi 4B. All devices are connected to the local network via appropriate Wi-Fi bands to simulate channel occupancy. The final data was collected from a real client, reflecting the performance of one device under resource competition. Due to limited actual devices, in cases of higher simulated device counts, a Raspberry Pi 5 simulated two devices. For the experiment validating the viewpoint change module, data was collected from an iPhone 15 Pro, which can provide real-time pose data to the program.

6.2. Evaluation Metrics

We use the following metrics to assess system performance:

- **Accuracy:** Evaluates detection performance. Our accuracy calculation method follows that of MOTA, considering both missed detections and false positives. The IoU threshold is set to 0.75.
- **End-to-end latency:** Since rendering, local detection, and remote detection are performed on parallel threads, we report the latency of edge server detection, which dominates overall system performance in most scenarios.
- **Number of packages:** The average number of packages sent per video, used to measure the impact of the tracking algorithm on decision-making.
- **Total package size:** The total size of data offloaded to the edge server.

6.3. Performance Overview

Fig. 12 shows an example output of MacEdge. The two objects on the left are newly detected targets. Because they have been observed for fewer than four frames, they are continuously offloaded to allow the tracker to stabilize. The object in the lower left corner is turning, and it is difficult for the tracker to predict its next frame result, so it is calibrated. The object on the right is offloaded at regular intervals.

As illustrated in Fig. 13, across different frame rates, single-object offloading methods consistently outperform both local-only inference and real-time ROI encoding in terms of accuracy, regardless of the local backend or whether

tracking is enabled. Compared to local inference, MacEdge improves accuracy by over 100% at all frame rates. Compared to ROI offloading, the gains are 79.8%, 34.9%, and 137.3% at 30 FPS, 60 FPS, and 120 FPS, respectively. Notably, ROI encoding degrades sharply at 120 FPS, where false positives increase significantly. With shorter inter-frame intervals, motion cues become weaker and the tracker must extrapolate more aggressively, which amplifies ROI misalignment and error accumulation, ultimately harming detection reliability. In contrast, MacEdge maintains accuracy close to the edge-server baseline across frame rates, indicating that dynamic offloading introduces little additional error.

Fig. 14 further validates this conclusion on VisDrone. The left part reports accuracy against human annotations, reflecting absolute detection quality, while the right part reports accuracy against the teacher outputs from the remote edge-server model, which isolates the accuracy loss induced by offloading decisions. Under both evaluation protocols, MacEdge consistently achieves the best accuracy and remains the closest to the remote baseline across frame rates, whereas ROI shows a widening gap at high FPS, consistent with the observed increase in false positives.

In terms of latency, as shown in Fig. 15, MacEdge achieves the lowest end-to-end delay at all frame rates under all three network settings, and its average latency always remains below the corresponding frame interval, ensuring outputs can be produced at the target FPS. Compared to Edge Only, MacEdge reduces latency by 39.8%, 49.9%, and 70.2% in the 2.4 GHz Wi-Fi environment, and by 32.9%, 53.0%, and 65.1% in the 5 GHz Wi-Fi environment at three frame rates, respectively. Under the 5G cellular network, the corresponding reductions are 17.5%, 33.3%, and 38.8% at different frame rates, indicating that MacEdge remains effective even when cellular links introduce higher and more variable transmission delay. In contrast, without local inference, the larger offloaded payload leads to higher latency than MacEdge, for example, at 60 FPS, Edge Only is 22.1%,

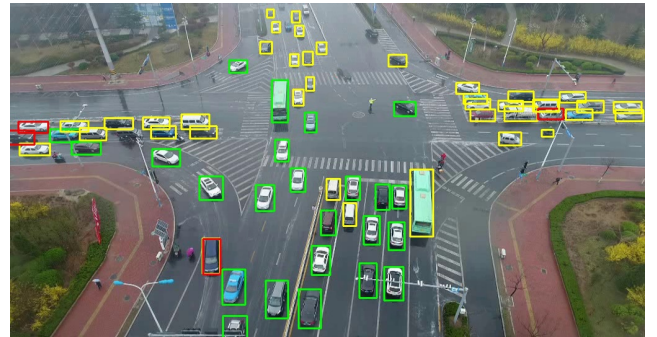


Fig. 12. An example of the results output by the MacEdge framework. The green part is the local model detection part, the yellow part is the result of the local tracker, and the red part is the result of the offload part. The overlap of yellow and red in the lower left corner indicates that the tracker is being calibrated.

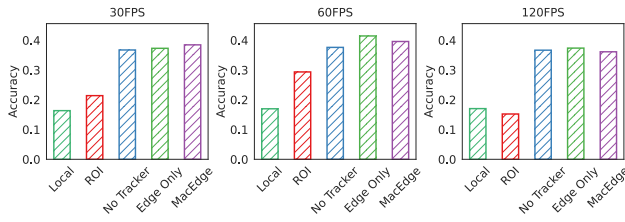


Fig. 13. Accuracy of different methods on the UAV Benchmark M.

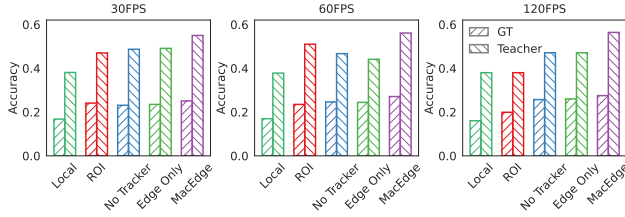


Fig. 14. Accuracy of different methods on the VisDrone dataset.

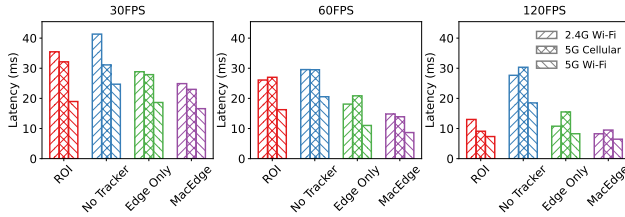


Fig. 15. Average end-to-end delay for different methods at different frame rates.

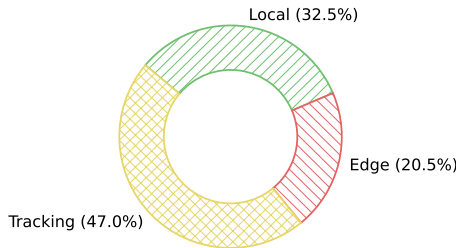


Fig. 16. In the M0203 video (120FPS), the proportion of detection results from different sources. Remote detection results account for only about one-fifth, and nearly half of the results come from trackers.

27.2%, and 50.0% higher than MacEdge under 2.4 GHz Wi-Fi, 5 GHz Wi-Fi, and 5G cellular, respectively.

An analysis of total package size and package count shown in Fig. 17 can further explain these results. Compared to the ROI method, the dynamic offloading scheme, with its finer granularity, adopts a strategy of more frequent, smaller offloads, meaning that it frequently offloads poorly tracked objects. In contrast, ROI's frame-based offloading incurs much higher transmission and inference costs per event. Given comparable latency requirements, tracker calibration is much less frequent with the ROI method.

These results also explain why, for high frame-rate videos, dynamic offloading yields lower latency without a significant drop in accuracy. For 30 FPS videos, edge only inference offloaded 1028 packages (consistent with the frame rate, as every frame was offloaded), while MacEdge offloaded only 828 packages, meaning about one fifth of frames had no objects to offload. For 120 FPS, pure edge-side inference offloaded 4092 packages, while MacEdge offloaded just 1866 times. More than half the frames required no offloading, as local tracking results were sufficient, resulting in zero remote inference latency for those frames. Notably, total offload count increased with the dynamic method, but overall data size decreased. Besides parameter differences, another reason is that frame-to-frame changes are smaller at high FPS, reducing the likelihood of triggering full-frame offloading, which usually contributes more to data size and cost. For example, for video M0203 in the dataset, at 30 FPS, full-frame offloading occurred 63 times; at 120 FPS, only 37 times. As shown in Fig. 16, in the case of 120 frames, most of the detection results come from local, and the proportion of remote results in all results is very small. This means that the tracker's estimation is valid for most targets.

6.4. Impact of different device counts and network bandwidth

We also analyzed system latency under varying numbers of devices and different network bandwidths using 60 FPS videos. To avoid inaccuracies from frequent video switching on the client, only videos longer than 30 seconds were used.

Since ROI encoding was not designed for batch processing and significantly lags behind other methods, it was excluded from multi-device latency comparisons. With three concurrent devices, its latency already exceeds 40 ms. Therefore, we did not put its delay into the Fig. 18. The number of devices mainly affects the latency of the inference phase. Too many requests cause requests to be blocked in the queue and need to be processed in a queue. From the experimental results, we can see that when we adjust the size of the canvas to twice the original size, the latency has dropped significantly. We also updated the size of the model, and the final accuracy has only a slight difference. This shows that when the number of requests is huge, our method can quickly improve its throughput by simply increasing the model size.

Network bandwidth primarily affects end-to-end latency at the transmission layer. Experimental results show that bandwidth requirements scale with offloaded data size. ROI encoding, which is not shown in the Fig. 19, reaches a latency of 845 ms at just 10 Mbps. From Fig. 20, we can see that the package size of the ROI method is much larger than that of the other three methods, which leads to very poor performance under limited bandwidth. The latency comparison of the other three methods is basically consistent with the amount of data they offload. In this scenario, our method can meet the latency requirements under a bandwidth of 40 Mbps and the amount of data offloaded is also the least.

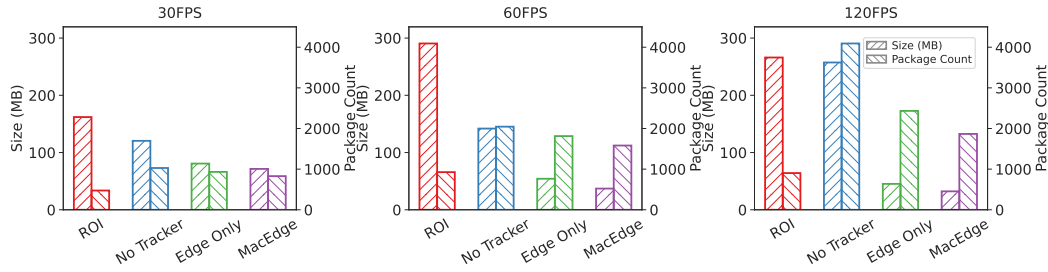


Fig. 17. Package size (left) and average number of packages (right) when analyzing a single video at different frame rates for different methods.

6.5. Design benefit analysis of different modules

6.5.1. Percentage of running time for different modules

Fig. 21 shows that the main components of the overall runtime are local detection, offloading (including merging,

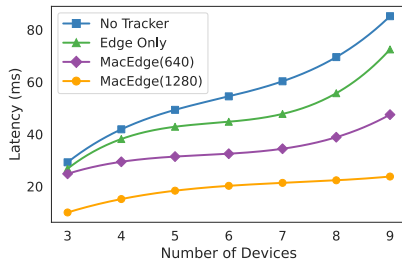


Fig. 18. Differences in latency performance for different numbers of devices, where MacEdge gives two different canvas sizes, using the model corresponding to the input size for inference.

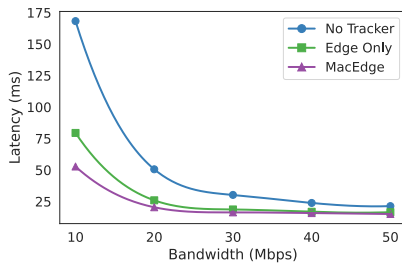


Fig. 19. Performance of latency at different bandwidths, the latency of ROI coding is too high and is not shown for comparison in the figure.

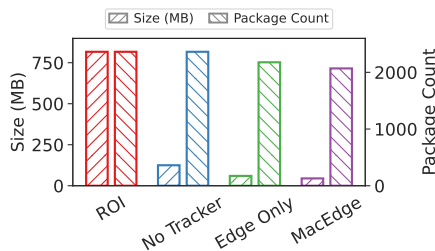


Fig. 20. Differences in package size and number between methods in bandwidth limiting experiments.

Table 2

Impact of different cropping methods on the integrated performance of the system

	Fixed Cropping	Dynamic Cropping
Package Size (MB)	5.23	3.89
End to End Latency (ms)	7.988	6.191
Accuracy	0.3855	0.3958

encoding, and remote detection), and rendering. Since these modules are decoupled and designed to run in parallel, the system still meets real-time inference requirements, even though the total running time across all modules exceeds the delay requirement. This demonstrates the necessity of decoupling and parallelizing different tasks for real-time analysis. Additionally, our tracking algorithm is highly efficient, taking less than 1 ms and having negligible impact on performance, which makes per-frame operation feasible.

6.5.2. Impact of Dynamic Cropping

We evaluated the effect of the dynamic cropping module by setting the crop size to a fixed value of twice the width and height of the original detection box. As shown in Table 2, the offloading strategy using fixed-size cropping performed worse in all respects. Dynamic cropping improved accuracy by 2.6%, while reducing total data size and end-to-end latency by 25.6% and 22.49%, respectively. We found that this reduction in offloaded data is mainly due to stationary or uniformly moving objects, for which the dynamically determined cropping size is significantly smaller than that in the fixed cropping strategy. For fast-moving objects, using a

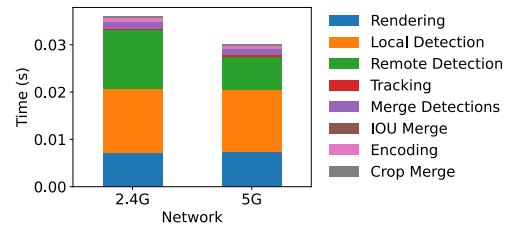


Fig. 21. Running time of different modules of the system, where local detection, offloading process and rendering can be executed in parallel.

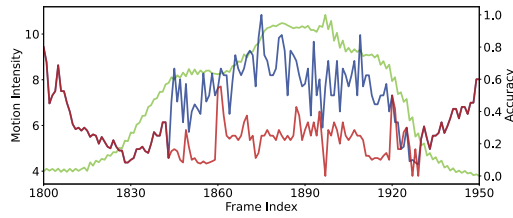


Fig. 22. Changes in the magnitude of viewpoint change (green) and accuracy (red and blue) over a range of frame intervals for a segment of moving viewpoints, with blue indicating a fixed offloading strategy and red indicating a dynamic offloading strategy based on the magnitude of viewpoint change.

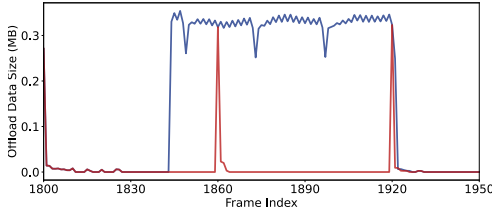


Fig. 23. The size of the offloaded package within the frame interval of a moving view can reflect the aggressiveness of offload scheduling.

larger cropping window can help reduce the missed detection rate.

6.5.3. Content-Based Full-Frame Offloading Decision

To evaluate the benefit of our approach, we removed the dynamic full-frame offloading module and used only a fixed full-frame offloading strategy as a baseline. For the dynamic method, decisions were based on gyroscope data provided by the iOS CoreMotion library, and the output of our motion intensity evaluation program was also used for validation. As shown in Fig. 22 and Fig. 23, when the set threshold was exceeded, our scheduler actively initiated offloading to overcome tracking difficulties caused by camera motion, resulting in overall accuracy that consistently outperformed fixed-interval offloading strategies. **It can be seen from the offloaded package size in Fig. 22 that after removing the decision module for viewpoint change, even though the accuracy is severely reduced, the package size did not change at all.** This is because the sudden change in viewpoint makes almost all trackers invalid and unable to work normally as expected. Even if the specified area is cropped for offloading, the target cannot be detected.

7. Conclusion

In this paper, we propose MacEdge, a real-time video analytics framework that leverages tracking algorithms to optimize offloading decisions and enhances edge server throughput through sub-image stitching. The modular design of the framework makes it suitable for deployment across a wide range of devices and application scenarios. Experimental results on the **UAV Benchmark M** demonstrate that, compared to fully local inference, MacEdge achieves more

than a 100% improvement in accuracy, while maintaining accuracy comparable to full edge-side inference. With the integration of a local inference module, MacEdge also meets inference latency requirements across different resolutions and network environments, demonstrating its practicality for real-time video analytics on edge devices.

The proposed method also holds significant potential for future extension. It can be adapted to support a wide range of real-time video analytics tasks beyond object detection, including target segmentation, depth estimation, and other complex scenarios. Owing to its lightweight design and low resource consumption, the framework is well suited for deployment on diverse edge devices such as UAVs, surveillance cameras, and augmented reality platforms. This flexibility further underscores the practicality and scalability of MacEdge in real-world applications.

References

- [1] He Xue, Dajiang Chen, Ning Zhang, Hong-Ning Dai, and Keping Yu. Integration of blockchain and edge computing in internet of things: A survey. *Future Generation Computer Systems*, 144:307–326, 2023.
- [2] Oshin Rawlley, Shashank Gupta, Kashish Mahajan, Aishna Shrivastava, and Esha Jain. Hs-giov: High-speed green internet of vehicles (iov) edge-assisted model for low-latency inference in autonomous driving. *Future Generation Computer Systems*, 169:107817, 2025.
- [3] Tao Zhang, Yan Zhao, Wenjing Jia, and Mu-Yen Chen. Collaborative algorithms that combine ai with iot towards monitoring and control system. *Future Generation Computer Systems*, 125:677–686, 2021.
- [4] Dongzhu Xu, Anfu Zhou, Guixian Wang, Huanhuan Zhang, Xiangyu Li, Jialiang Pei, and Huadong Ma. Tutti: coupling 5g ran and mobile edge computing for latency-critical video analytics. In *Proceedings of the 28th Annual International Conference on Mobile Computing And Networking, MobiCom '22*, page 729–742, New York, NY, USA, 2022. Association for Computing Machinery.
- [5] Xinyi Zhang, Haoran Xu, Chenyun Yu, and Guang Tan. PCTrack: Accurate Object Tracking for Live Video Analytics on Resource-Constrained Edge Devices. *IEEE Transactions on Circuits and Systems for Video Technology*, 35(5):3969–3982, May 2025.
- [6] Shuochao Yao, Jinyang Li, Dongxin Liu, Tianshi Wang, Shengzhong Liu, Huajie Shao, and Tarek Abdelzaher. Deep compressive offloading: Speeding up neural network inference by trading edge computation for network latency. In *Proceedings of the 18th Conference on Embedded Networked Sensor Systems*, pages 476–488, Virtual Event Japan, November 2020. ACM.
- [7] Luyang Liu, Hongyu Li, and Marco Gruteser. Edge Assisted Real-time Object Detection for Mobile Augmented Reality. In *The 25th Annual International Conference on Mobile Computing and Networking*, pages 1–16, Los Cabos Mexico, August 2019. ACM.
- [8] Xu Wang, Zheng Yang, Jiahang Wu, Yi Zhao, and Zimu Zhou. Edge-Duet: Tiling Small Object Detection for Edge Assisted Autonomous Mobile Vision. In *IEEE INFOCOM 2021 - IEEE Conference on Computer Communications*, pages 1–10, Vancouver, BC, Canada, May 2021. IEEE.
- [9] Shubham Chaudhary and Aryan Taneja. TileClipper: Lightweight Selection of Regions of Interest from Videos for Traffic Surveillance.
- [10] Romil Bhardwaj, Zhengxu Xia, Ganesh Ananthanarayanan, Junchen Jiang, Yuanchao Shu, Nikolaos Karianakis, Kevin Hsieh, Paramvir Bahl, and Ion Stoica. Ekya: Continuous learning of video analytics models on edge compute servers. In *19th USENIX Symposium on Networked Systems Design and Implementation (NSDI 22)*, pages 119–135, 2022.
- [11] Shoufei Han, Xiaojing Liu, MengChu Zhou, Kun Zhu, Liang Zhao, Aiiad Albeshri, and Abdullah Abusorrah. Joint Association, Deployment and Flight Trajectory Optimization for Multi-UAV-enabled

- Large-Scale Mobile Edge Computing. *IEEE Transactions on Mobile Computing*, pages 1–15, 2024.
- [12] Ross Girshick. Fast R-CNN. In *2015 IEEE International Conference on Computer Vision (ICCV)*, pages 1440–1448, December 2015.
 - [13] Chenguang Liu, Guangshuai Gao, Ziyue Huang, Zhenghui Hu, Qingjie Liu, and Yunhong Wang. YOLC: You Only Look Clusters for Tiny Object Detection in Aerial Images, June 2024.
 - [14] Mengqi Lei, Siqi Li, Yihong Wu, Han Hu, You Zhou, Xinhui Zheng, Guiguang Ding, Shaoyi Du, Zongze Wu, and Yue Gao. Yolov13: Real-time object detection with hypergraph-enhanced adaptive visual perception. *arXiv preprint arXiv:2506.17733*, 2025.
 - [15] Yifan Feng, Jiangang Huang, Shaoyi Du, Shihui Ying, Jun-Hai Yong, Yipeng Li, Guiguang Ding, Rongrong Ji, and Yue Gao. Hyper-yolo: When visual object detection meets hypergraph computation. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2024.
 - [16] Yuming Chen, Xinbin Yuan, Jiabao Wang, Ruiqi Wu, Xiang Li, Qibin Hou, and Ming-Ming Cheng. Yolo-ms: rethinking multi-scale representation learning for real-time object detection. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2025.
 - [17] Xuecheng Wu, Junxiao Xue, Liangyu Fu, Jiayu Nie, Danlei Huang, and Xinyi Yin. 3a-yolo: New real-time object detectors with triple discriminative awareness and coordinated representations. *arXiv preprint arXiv:2412.07168*, 2024.
 - [18] Abhijeet Pujara and Mamta Bhamare. DeepSORT: Real Time & Multi-Object Detection and Tracking with YOLO and TensorFlow. In *2022 International Conference on Augmented Intelligence and Sustainable Systems (ICAISS)*, pages 456–460, November 2022.
 - [19] Alex Bewley, Zongyuan Ge, Lionel Ott, Fabio Ramos, and Ben Upcroft. Simple online and realtime tracking. In *2016 IEEE International Conference on Image Processing (ICIP)*, pages 3464–3468, September 2016.
 - [20] Ruopeng Gao, Ji Qi, and Limin Wang. Multiple object tracking as id prediction. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pages 27883–27893, 2025.
 - [21] Kai Luo, Hao Shi, Sheng Wu, Fei Teng, Mengfei Duan, Chang Huang, Yuhang Wang, Kaiwei Wang, and Kailun Yang. Omnidirectional multi-object tracking. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pages 21959–21969, 2025.
 - [22] Biyi Fang, Xiao Zeng, Faen Zhang, Hui Xu, and Mi Zhang. FlexDNN: Input-Adaptive On-Device Deep Learning for Efficient Mobile Vision. In *2020 IEEE/ACM Symposium on Edge Computing (SEC)*, pages 84–95, San Jose, CA, USA, November 2020. IEEE.
 - [23] Seungyeop Han, Haichen Shen, Matthai Philipose, Sharad Agarwal, Alec Wolman, and Arvind Krishnamurthy. MCDNN: An Approximation-Based Execution Framework for Deep Stream Processing Under Resource Constraints. In *Proceedings of the 14th Annual International Conference on Mobile Systems, Applications, and Services*, pages 123–136, Singapore Singapore, June 2016. ACM.
 - [24] Hanling Wang, Tianyu Li, Mei Zhang, Qing Li, Huan Cui, Yong Jiang, and Zhenhui Yuan. Joint Configuration Optimization and GPU Allocation for Multi-Tenant Real-Time Video Analytics on Resource-Constrained Edge. *IEEE Transactions on Mobile Computing*, 24(2):794–811, February 2025.
 - [25] Kuntai Du, Qizheng Zhang, Anton Arapin, Haodong Wang, Zhengxu Xia, and Junchen Jiang. AccMPEG: Optimizing Video Encoding for Video Analytics, April 2022.
 - [26] Z. Jonny Kong, Qiang Xu, Jiayi Meng, and Y. Charlie Hu. AccuMO: Accuracy-Centric Multitask Offloading in Edge-Assisted Mobile Augmented Reality. In *Proceedings of the 29th Annual International Conference on Mobile Computing and Networking*, pages 1–16, Madrid Spain, October 2023. ACM.
 - [27] Zhengru Fang, Senkang Hu, Liyan Yang, Yiqin Deng, Xianhao Chen, and Yuguang Fang. Pib: Prioritized information bottleneck framework for collaborative edge video analytics. In *GLOBECOM 2024-2024 IEEE Global Communications Conference*, pages 5295–5300. IEEE, 2024.
 - [28] You Zhou, Changsheng You, and Kaibin Huang. Communication efficient cooperative edge ai via event-triggered computation offloading. *arXiv preprint arXiv:2501.02001*, 2025.
 - [29] Y. Kang, J. Hauswald, C. Gao, et al. Neurosurgeon: Collaborative intelligence between the cloud and mobile edge. In *Proceedings of the Twenty-Second International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2017.
 - [30] En Li, Liekang Zeng, Zhi Zhou, and Xu Chen. Edgent: An on-demand deep learning model co-inference framework for edge intelligence. In *Proceedings of the 16th Annual International Conference on Mobile Systems, Applications, and Services (MobiSys)*, 2018.
 - [31] Amir Erfan Eshratifar, Amirhossein Esmaili, and Massoud Pedram. Bottlenet: A deep learning architecture for intelligent mobile cloud computing services. In *Proceedings of the 2019 IEEE/ACM International Symposium on Low Power Electronics and Design (ISLPED)*, 2019.
 - [32] Jiawei Shao and Jun Zhang. Bottlenet++: An end-to-end approach for feature compression in device-edge co-inference systems. In *2020 IEEE International Conference on Communications Workshops (ICC Workshops)*, 2020.
 - [33] Matteo Mendula, Paolo Bellavista, Marco Levorato, and Sharon Ladrón de Guevara Contreras. A novel middleware for adaptive and efficient split computing for real-time object detection. *Pervasive and Mobile Computing*, page 102028, 2025.
 - [34] Yoshitomo Matsubara, Marco Levorato, and Francesco Restuccia. Split computing and early exiting for deep learning applications: Survey and research challenges. *ACM Computing Surveys*, 2022.
 - [35] Joseph Redmon, Santosh Divvala, Ross Girshick, and Ali Farhadi. You Only Look Once: Unified, Real-Time Object Detection. In *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 779–788, Las Vegas, NV, USA, June 2016. IEEE.
 - [36] Pengfei Zhu, Longyin Wen, Dawei Du, Xiao Bian, Heng Fan, Qinghua Hu, and Haibin Ling. Detection and tracking meet drones challenge. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 44(11):7380–7399, 2021.
 - [37] Huaizu Jiang, Deqing Sun, Varan Jampani, Ming-Hsuan Yang, Erik Learned-Miller, and Jan Kautz. Super SloMo: High Quality Estimation of Multiple Intermediate Frames for Video Interpolation. In *2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 9000–9008, Salt Lake City, UT, June 2018. IEEE.
 - [38] Chengzhi Wang and Peng Yang. Compression Metadata-assisted RoI Extraction and Adaptive Inference for Efficient Video Analytics, March 2025.